```
line.link {
      stroke-width: 1px;
      stroke: black;
      stroke-opacity: .5;
}
circle.node {
  fill: red;
  stroke: white;
  stroke-width: 1px;
}
circle.xy {
  fill: pink;
  stroke: black;
  stroke-width: 1px;
}
```

### 11.1.1 Creating random geodata

The first thing we need is a dataset with a thousand datapoints. Rather than using data from a pregenerated file, we'll invent it. One useful function available in D3 is `d3.range()`, which allows you to create an array of values. We'll use `d3.range()` to create an array of a thousand values. We'll then use that array to populate an array of objects with enough data to put on a network and on a map. Because we're going to put this data on a map, we need to make sure it's properly formatted geoJSON, as in the following listing, which uses the `randomCoords()` function to create triangles.

**Listing 11.3 Creating sample data**

```
var sampleData = d3.range(1000).map(function(d) {
   var datapoint = {};
   datapoint.id = "Sample Feature " + d;
   datapoint.type = "Feature";
   datapoint.properties = {};
   datapoint.geometry = {};
   datapoint.geometry.type = "Polygon";
   datapoint.geometry.coordinates = randomCoords();
   return datapoint;
});

function randomCoords() {
  var randX = (Math.random() * 350) - 175;
  var randY = (Math.random() * 170) - 85;
  return [[[randX - 5,randY],[randX,randY - 5],
        [randX - 10,randY - 5],[randX - 5,randY]]];
};
```

**d3.range creates an array that we immediately map to an object array.**

**Each datapoint is an object with the necessary attributes to be placed on a map.**

**Draws a triangle around each random lat/long coordinate pair**

After we have this data, we can throw it on a map like the one we first created in chapter 7. In the following listing we use the world.geojson file from chapter 7, so that we have some context for where the triangles are drawn.